Instituto Politécnico Superior “Gral. San Martín”
Departamento de Informática
Técnico/a en Informática Profesional y Personal
Docente: Lic. Muzzulini Regina

TEORÍA DE GRAFOS · 6TO AÑO

Árboles

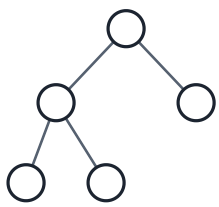
Grafos conexos y sin ciclos: su definición, propiedades, aplicaciones y los algoritmos que se construyen sobre ellos.

1. Definición · 2. Teorema de equivalencias · 3. Aplicaciones · 4. Árboles genealógicos · 5. Códigos de Huffman · 6. Árbol de expansión · 6.1 DFS · 6.2 BFS · 6.3 Ejemplo · 7. Árbol de expansión mínimo · 7.1 Prim · 7.2 Kruskal · 7.3 Algoritmo goloso · 8. Árboles con raíz · 8.1 Binarios · 8.2 Binarios completos · 8.3 m-arios · 8.4 m-arios completos · 8.5 Equilibrado · 9. Árboles binarios · 10. Árbol binario de búsqueda · 11. Recorridos · 12. Árboles de expresión aritmética · 13. Árboles de decisión · Bibliografía

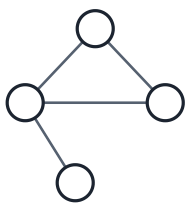
1. Definición de árbol

Definición: $T = (V, E)$ es un **árbol** si para todo par de nodos $u, r \in V$ existe un **único** camino $u–r$.

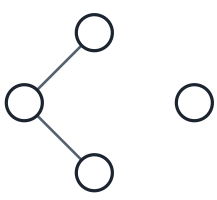
Árbol con raíz: es un par (T, r) donde $T = (V,E)$ es un árbol y $r \in V$ es un nodo distinguido.



G_1 — es árbol



G_2 — no es árbol (tiene un ciclo)



G_3 — no es árbol (no es conexo)

Pregunta: ¿se puede construir un árbol con 5 nodos y 6 aristas? ¿Con 5 aristas? ¿Con 4? ¿Con 3?

2. Teorema de equivalencias

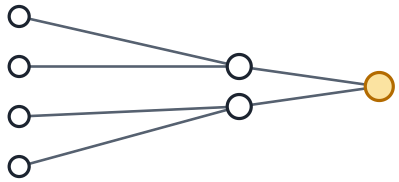
Sea `T = (V, E)` . Las siguientes afirmaciones son equivalentes:

> TEOREMA
>
> 1. T es árbol.
> 2. T es conexo y acíclico.
> 3. T es simple, conexo y `|E| = |V| − 1` .
> 4. T es acíclico y `|E| = |V| − 1` .

3. Aplicaciones

I. Torneos

Brasil, Yugoslavia, Rusia y Francia juegan semifinales (T_1 y T_2) y el ganador de cada una juega la final: la llave del campeonato es un árbol.



Brasil / Yugoslavia / Rusia / Francia → T_1, T_2 → Campeón

II. Organigramas

Director de escuela de Cs. Exactas → Director de dpto. de matemática, Director de dpto. de física, Director de dpto. de computación: una jerarquía es un árbol con raíz.

4. Árboles genealógicos

Sea `(T, r)` un árbol con raíz.

- **Nivel** de un nodo *u*: la longitud del camino *u*–*r*.
- **Altura** de `(T,r)` : el máximo de los niveles de sus nodos.

- **Ancestros** de *y*: todos los nodos del camino *r–y* (excepto *y*).
- **Descendiente:** *w* es descendiente de *y* si *y* es ancestro de *w*.
- **Padre** de *y*: el nodo anterior a *y* en el camino *r–y*.
- **Hijo** de *x*: *u* es hijo de *x* si *x* es el padre de *u*.
- **Hermanos:** *u* y *t* son hermanos si tienen el mismo padre.
- **Hoja** (o terminal): *u* es hoja si no tiene hijos.
- **Subárbol** de *T* con raíz *z*: el par `(T_z, z)`, donde `T_z` es el subgrafo de T inducido por *z* y sus descendientes.

5. Códigos de Huffman

Códigos de longitud fija y de longitud variable

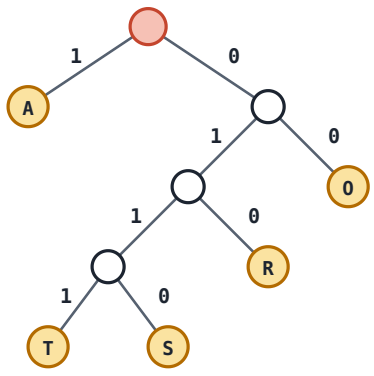
La forma más común de representar caracteres en una computadora es con cadenas de bits de **longitud fija**. Por ejemplo, el código **ASCII** (*American Standard Code for Information Interchange*) usa 7 bits por carácter:

Carácter	A	B	C	1	2	!	*
Código ASCII	100 0001	100 0010	100 0011	011 0001	011 0010	010 0001	010 1010

Los **códigos de Huffman** representan los caracteres con cadenas de bits de **longitud variable**: la idea es usar cadenas *cortas* para los caracteres que se usan con más frecuencia y cadenas *largas* para los de uso menos frecuente. Así, en general, un texto o un programa ocupa menos espacio que si se codificara con ASCII.

Un código de Huffman es un árbol con raíz

Un código de Huffman se define mediante un **árbol con raíz**: los caracteres están en las **hojas**, y cada arista se etiqueta con un bit (**1** si va hacia la izquierda, **0** si va hacia la derecha). El código de un carácter es la sucesión de bits del camino que va desde la raíz hasta su hoja.



Árbol de un código de Huffman

Carácter	Código	Longitud
A	1	1
O	00	2
R	010	3
S	0110	4
T	0111	4

La longitud del código de un carácter es el **nivel** de su hoja en el árbol.

Decodificación

Para decodificar una cadena de bits se empieza en la raíz y se baja por el árbol siguiendo los bits (1 a la izquierda, 0 a la derecha) hasta llegar a una hoja: ahí se lee un carácter y se vuelve a empezar desde la raíz.

> EJEMPLO
>
> Decodificar `01010111` : `010` → R, `1` → A, `0111` → T. La cadena representa **RAT**.

Como los caracteres están **sólo en las hojas**, ningún código es el comienzo (prefijo) de otro, y por eso cualquier cadena de bits se decodifica de **una única manera**, aunque los códigos tengan longitudes distintas. Además, el árbol de un código de Huffman es un **árbol binario completo**: cada nodo interno tiene exactamente 2 hijos.

El problema

> PROBLEMA
>
> Dado un alfabeto con frecuencias asociadas a cada carácter, construir el código de Huffman que represente las cadenas de caracteres en **espacio mínimo**.

Si el carácter *i* tiene frecuencia f_i y su hoja está en el nivel n_i, el texto ocupa $\Sigma\, f_i \cdot n_i$ bits: se busca el árbol que minimice esa suma.

Algoritmo de Huffman

Construye un código de Huffman óptimo a partir de la tabla de frecuencias. La salida es un árbol con raíz cuyas hojas se etiquetan primero con las **frecuencias** y las aristas con bits; al final se reemplaza cada frecuencia por un carácter que tenga esa frecuencia.

```
Entrada: sucesión f de n frecuencias, n ≥ 2
Salida:  árbol con raíz que define un código de Huffman óptimo

huffman(f, n):
  si n = 2:
     sean f₁ y f₂ las frecuencias
     T = árbol con raíz, hijo izquierdo f₁ (arista 1) e hijo derecho f₂ (arista
0)
     devolver T
  sean fᵢ y fⱼ las dos frecuencias más pequeñas
  sustituir fᵢ y fⱼ en la lista f por fᵢ + fⱼ
  T = huffman(f, n − 1)
  sustituir en T un vértice etiquetado fᵢ + fⱼ por un árbol con
     hijo izquierdo fᵢ (arista 1) e hijo derecho fⱼ (arista 0)
  devolver T
```

(La demostración de que el código construido es óptimo no se incluye acá.)

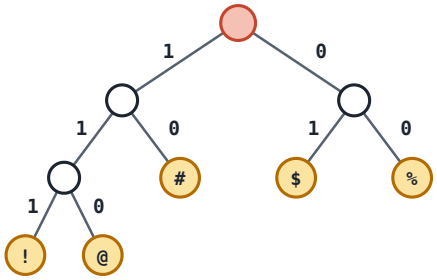
Ejemplo

Frecuencias: `! → 2` `@ → 3` `# → 7` `$ → 8` `% → 12`

Primero se reemplazan, una y otra vez, las dos frecuencias más pequeñas por su suma, hasta quedarse con dos elementos:

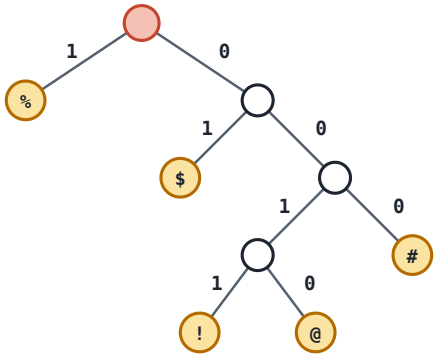
```
2, 3, 7, 8, 12   →   (2+3), 7, 8, 12
5, 7, 8, 12      →   (5+7), 8, 12
8, 12, 12        →   (8+12), 12
12, 20
```

Después se construye el árbol "hacia atrás": se parte de la raíz con hijos 12 (arista 1) y 20 (arista 0); el 20 se reemplaza por un árbol con hijos 8 y 12 (porque 20 = 8 + 12); uno de los 12 se reemplaza por un árbol con hijos 5 y 7; y el 5 por uno con hijos 2 y 3. Por último, cada frecuencia se reemplaza por su carácter.



Árbol 1 — altura 3

Carácter	!	@	#	$	%
Código	111	110	10	01	00



Árbol 2 — altura 4

Carácter	!	@	#	$	%
Código	0011	0010	000	01	1

El árbol de Huffman no es único. Cuando hay que reemplazar un 12 por un árbol con hijos 5 y 7, en el árbol hay dos vértices con etiqueta 12 y se puede elegir cualquiera. Si se elige uno se obtiene el árbol 1; si se elige el otro, el árbol 2. Los dos son óptimos: codifican un texto con esas frecuencias en **exactamente el mismo espacio**.

> COSTO
> Árbol 1: 2·3 + 3·3 + 7·2 + 8·2 + 12·2 = **69 bits**. Árbol 2: 2·4 + 3·4 + 7·3 + 8·2 + 12·1 = **69 bits**.
> Con ASCII, los 32 caracteres (2+3+7+8+12) ocuparían 7·32 = **224 bits**.

Nodos del árbol de Huffman

Como el árbol de Huffman es un árbol binario completo, si el alfabeto tiene *n* caracteres (*n* hojas), el árbol tiene `n − 1` nodos internos y `2n − 1` nodos en total.

6. Árbol de expansión

Consideremos el problema de determinar un subgrafo `T` de un grafo `G` de modo que `T` sea un árbol con todos los vértices de `G` ; es decir, un **árbol de expansión** (también llamado árbol generador). Los métodos para determinar árboles de expansión se pueden aplicar también a otros problemas.

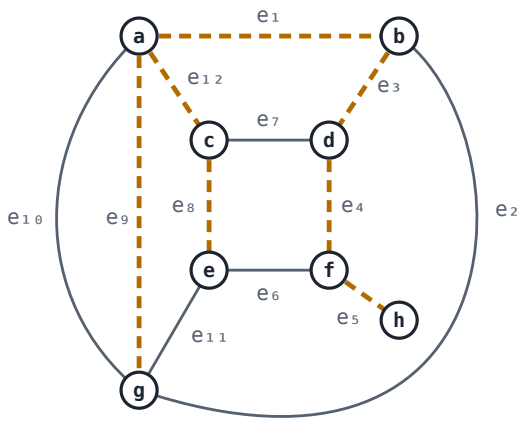
Definición: un árbol `T` es un **árbol de expansión** de un grafo `G` si `T` es un subgrafo de `G` que contiene a todos los vértices de `G` . Si `G = (V, E)` y `T = (V', E')` , esto significa que `V' = V` y `E' ⊆ E` .

> PROPIEDAD
>
> Un grafo `G = (V, E)` tiene un árbol de expansión `T = (V', E')` si y sólo si `G` es conexo.

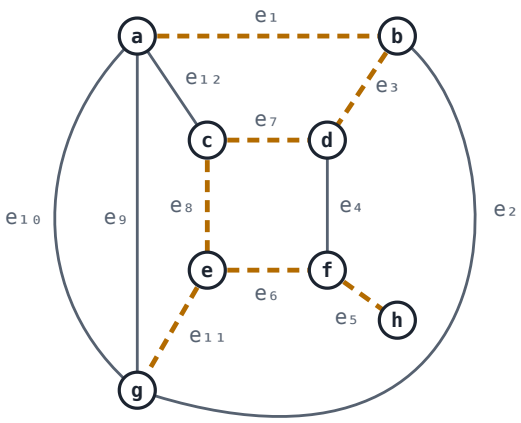
> EJEMPLO
>
> Si G no es conexo (por ejemplo, un pentágono más un vértice suelto), G no tiene árbol de expansión. Por eso, para decidir si un grafo es conexo se puede intentar "construir" un árbol de expansión.

Un grafo puede tener varios árboles de expansión

Un árbol de expansión del grafo `G` de la figura de la izquierda aparece en líneas punteadas. En general, un grafo tendrá varios árboles de expansión posibles: en la figura de la derecha está marcado otro árbol de expansión del mismo grafo.



Árbol de expansión 1: e_1, e_3, e_4, e_5, e_8, e_9, e_{12}



Árbol de expansión 2: e_1, e_3, e_5, e_6, e_7, e_8, e_{11}

Las aristas e_9 y e_{10} son paralelas (ambas unen a y g). Los dos árboles tienen 8 vértices y 7 aristas.

Algoritmos para encontrar un árbol de expansión

Vamos a ver dos algoritmos para encontrar un árbol de expansión: la **búsqueda en profundidad** (DFS) y la **búsqueda en anchura** (BFS). Lo primero que hay que considerar es su **entrada**: un grafo conexo `G` y un **orden entre los vértices**; si no se indica explícitamente, se usa el orden usual numérico o lexicográfico. Su **salida** es un árbol de expansión (o recursión) y BFS usa una **cola**.

Construir un árbol de expansión equivale a recorrer el grafo con un método de búsqueda. Ambos parten de un nodo raíz `v1` y usan un **orden fijo de los vértices** `v1, v2, ..., vn` para desempatar cuando hay varias opciones.

6.1 Búsqueda en profundidad (DFS)

`DFS` — *Depth First Search* (búsqueda en profundidad): se avanza por una rama hasta el final antes de retroceder y explorar otra.

```
Entrada: G = (V, E) conexo, con vértices ordenados v₁, v₂, ..., vₙ
Salida:  T = (V(T), E(T)) árbol de expansión de G, con raíz v₁

Inicio:
  V(T) = {v₁},   E(T) = ∅
  w = v₁

Mientras verdadero hacer:
  Mientras exista (w, v) ∈ E con v ∉ V(T) hacer:
    elegir vₖ con k mínimo tal que (w, vₖ) ∈ E y vₖ ∉ V(T)
    E(T) ← E(T) ∪ {(w, vₖ)}
    V(T) ← V(T) ∪ {vₖ}
    w ← vₖ                          // avanzar en profundidad
  Si w = v₁ entonces devolver T
  w ← padre de w en T               // retroceder (backtracking)
```

Como en cada paso se vuelve por una arista hacia la raíz cuando no se puede seguir avanzando, a la búsqueda en profundidad también se la llama **búsqueda con retroceso** (*backtracking*).

6.2 Búsqueda en anchura (BFS)

`BFS` — *Breadth First Search* (búsqueda en anchura): se exploran primero todos los vecinos del nodo actual antes de avanzar al siguiente nivel.

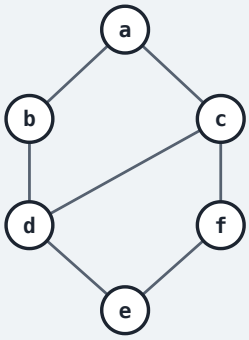
```
Entrada: G = (V, E) conexo, con vértices ordenados v₁, v₂, ..., vₙ
Salida:  T = (V(T), E(T)) árbol de expansión de G, con raíz v₁

Inicio:
  S = (v₁)                          // lista ordenada: nodos del nivel actual
  V(T) = {v₁},   E(T) = ∅

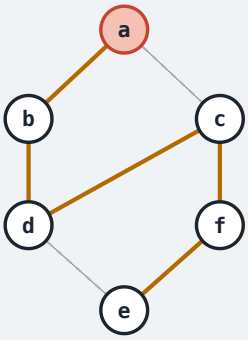
Mientras verdadero hacer:
  Para cada x ∈ S, en orden, hacer:
    Para cada y ∉ V(T), en orden, hacer:
      Si (x, y) ∈ E entonces:
        E(T) ← E(T) ∪ {(x, y)}
        V(T) ← V(T) ∪ {y}
  Si no se agregó ninguna arista entonces devolver T
  S ← hijos de los nodos de S, en el orden original   // pasar al nivel siguiente
```

6.3 Ejemplo y observaciones

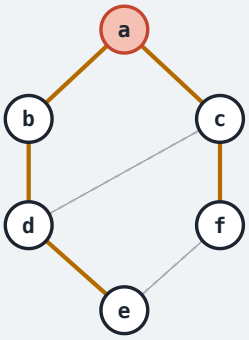
EJEMPLO — orden de vértices a, b, c, d, e, f · raíz a



G



DFS: a → b → d → c → f → e



BFS: nivel 1 {b, c} · nivel 2 {d, f} · nivel 3 {e}

DFS: desde *a* se va a *b*, luego a *d*, *c*, *f* y *e*. Desde *e* ya no hay vecinos nuevos, así que se retrocede (f, c, d, b) hasta volver a *a* y el algoritmo termina.

BFS: primero se agregan todos los vecinos de *a* (b y c); después los vecinos nuevos de b (d) y de c (f); por último el de d (e).

OBSERVACIÓN

Cualquiera de los dos sirve para decidir si un grafo G con n vértices es conexo: G es conexo si y solo si el árbol T que se obtiene tiene n vértices.

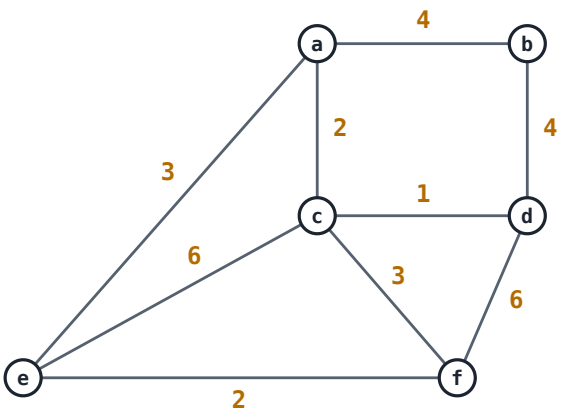
OBSERVACIÓN

En el árbol que construye BFS, el nivel de cada vértice es la longitud del camino más corto (en cantidad de aristas) desde la raíz.

Implementación habitual: DFS usa una **pila** (o recursión) y BFS usa una **cola**.

7. Árbol de expansión mínimo

El grafo con pesos de la siguiente figura muestra seis ciudades y los costos de construcción de carreteras entre ciertos pares de ellas. Queremos construir el sistema de carreteras de menor costo que una estas seis ciudades.



G — vértices: ciudades · aristas: carreteras posibles · peso: costo de construcción

La solución a este problema se puede representar mediante un subgrafo el cual debe ser un árbol de expansión porque:

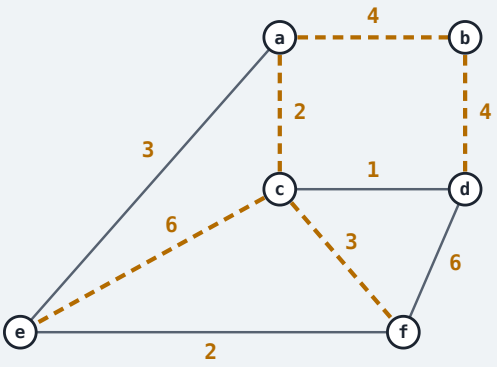
- debe tener todos los vértices (cada ciudad debe estar en el sistema de carreteras),
- debe ser conexo (para que podamos llegar de una ciudad a cualquier otra),
- debe tener un único camino simple entre cada par de vértices (para que sea de costo mínimo).

Así, lo que estamos necesitando es un árbol de expansión cuya suma de pesos sea mínima. Un árbol de esas características se llama **árbol de expansión mínimo**.

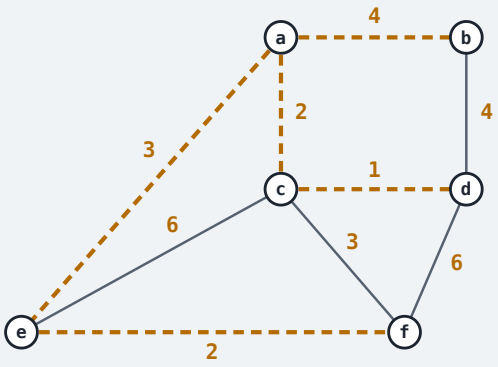
Formalmente: sea `G = (V, E)` con un vector de costos `c ∈ R₊^|E|` (un costo por cada arista). El costo de un árbol de expansión `T` es `c(T) = Σ c_e` (suma sobre las aristas de T), y se busca el árbol de expansión de menor costo.

EJEMPLO

Un mismo grafo puede tener distintos árboles de expansión con distinto costo. En el grafo de las ciudades hay, por ejemplo, uno de costo 19 y otro de costo 12; el objetivo es encontrar el de menor costo.



Árbol de expansión de costo 19



Árbol de expansión de costo 12

Para resolver esto, vamos a ver dos algoritmos, los cuales difieren en la forma en la que se construye el árbol. Uno comienza, como los anteriores, con un solo vértice y sin aristas, y va agregando vértices y aristas. En el otro, el árbol comienza con todos los vértices y sólo va agregando aristas; la dificultad que este último trae es que se debe analizar que la arista agregada no forme circuito. Ambos, al igual que los dos anteriores, toman un orden de vértices.

Veremos cada uno de estos algoritmos en detalle: uno se denomina **Prim** y, el otro, **Kruskal**.

7.1 Algoritmo de Prim

Construye un árbol de expansión mínimo agregando, en cada paso, la arista más barata que conecta el árbol parcial con un vértice nuevo.

```
Entrada: G = (V, E) conexo, c ∈ R+^|E| = (c_e1, c_e2, ..., c_em)
Salida:  T = (V(T), E(T)) árbol de expansión tal que
         c(T) = Σ c_e = mín{ c(T') : T' árbol de expansión de G }

Inicio:
  elegir s ∈ V
  V(T) = {s},   E(T) = ∅

Para i = 1 hasta m−1 hacer:
  encontrar (u, v) ∈ E tal que u ∈ V(T), v ∉ V(T)
           y c(u, v) = mín{ c(r, s) : r ∈ V(T), s ∉ V(T) }
  E(T) ← E(T) ∪ {(u, v)}
  V(T) ← V(T) ∪ {v}
```

(El teorema que garantiza que el algoritmo de Prim siempre da el árbol de costo mínimo se demuestra en la práctica; acá solo se enuncia y se aplica.)

7.2 Algoritmo de Kruskal

Foresta de G: un subgrafo de G sin ciclos (cada componente conexa es un árbol).

Dado $G = (V,E)$ y $c \in R_+^{|E|}$, se busca la foresta F de máximo costo. El algoritmo goloso aplicado a este problema: en cada iteración, se incorpora a F la arista de máximo peso que no forma un ciclo con los ya elegidos.

Si G es conexo, la máxima foresta es el máximo árbol de expansión.

¿Sirve este algoritmo para hallar el árbol de expansión mínimo? Sí: invirtiendo los costos —tomando $M = \max\{c_e\}$ y $c'_e = M - c_e \geq 0$ — y buscando el árbol de expansión máximo con los nuevos costos, se obtiene el árbol de expansión mínimo con los costos originales. En cada iteración se incorpora la arista de **mínimo** peso que no forma ciclos con los anteriores: este es el **algoritmo de Kruskal**.

7.3 Algoritmo goloso (greedy)

¿Por qué "goloso"? El nombre viene del inglés *greedy* (codicioso, glotón). Un algoritmo goloso construye la solución paso a paso y, en cada paso, **agarra lo que parece mejor en ese momento** —el "bocado más

grande"— sin mirar hacia adelante y **sin volver atrás** para deshacer una elección ya hecha. Como un goloso frente a una bandeja: toma el pedazo más grande que tiene a mano, sin pensar si después le conviene otro.

Prim y Kruskal son algoritmos golosos: en cada paso eligen la arista más barata que se puede agregar.

```
Entrada: elementos 1, 2, ..., n con costos c₁ ≥ c₂ ≥ ... ≥ cₙ
         y un criterio para decidir si un conjunto es una solución válida
Salida:  F solución válida (de costo "grande")

Inicio: F = ∅
Para i = 1 hasta n hacer:
  Si F ∪ {i} sigue siendo válida entonces
    F ← F ∪ {i}          // se queda con el mejor disponible y no lo suelta
Devolver F
```

¿El algoritmo goloso siempre da la solución correcta? No. En algunos problemas, por haberse "tentado" al principio con una opción muy buena, el goloso ya no puede tomar otras que, juntas, valían más, y termina con una solución peor que la óptima.

Para el árbol de expansión mínimo, en cambio, la estrategia golosa sí da siempre la solución óptima.

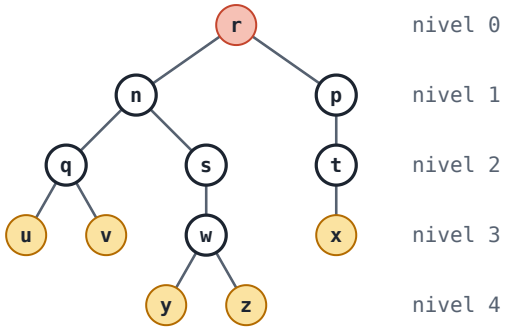
8. Árboles con raíz

En la sección 4 vimos la terminología de los árboles con raíz (nivel, altura, padre, hijo, ...). Ahora los definimos con más precisión, porque son la base de los árboles binarios y de las estructuras que siguen.

Definición: un **árbol dirigido** es un grafo dirigido cuyo grafo no dirigido asociado (el que se obtiene al borrar las flechas) es un árbol. Un árbol dirigido es un **árbol con raíz** si existe un único vértice `r`, llamado la **raíz**, tal que su grado de entrada es `ge(r) = 0` y, para todos los demás vértices *v*, el grado de entrada es `ge(v) = 1`.

Es decir: las flechas "bajan" desde la raíz y cada vértice, salvo la raíz, recibe una sola flecha: la de su padre. Los árboles con raíz se dibujan con la raíz arriba y las aristas dirigidas de arriba hacia abajo (del nivel superior al inferior), por lo que no es necesario dibujar las flechas.

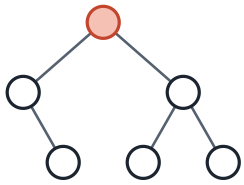
El **grado de salida** de un vértice *v*, `gs(v)`, es la cantidad de aristas que salen de *v*, es decir, la cantidad de hijos de *v*. Si `gs(v) = 0`, *v* es una **hoja** (o vértice terminal); todos los demás vértices son **vértices internos** (o nodos de ramificación).



Árbol con raíz *r*. Hojas (en amarillo): u, v, x, y, z. Altura: 4.

8.1 Árboles binarios

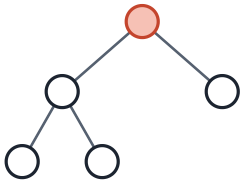
Definición: un **árbol binario** es un árbol con raíz donde cada vértice tiene a lo sumo 2 hijos, es decir, `gs(v) = 0, 1 ó 2`, que se distinguen como hijo "izquierdo" e hijo "derecho".



Árbol binario: cada vértice tiene 0, 1 ó 2 hijos

8.2 Árboles binarios completos

Un árbol binario es **completo** si `gs(v) = 0` ó `gs(v) = 2` para todo vértice *v*: cada vértice tiene 2 hijos o ninguno.



Árbol binario completo

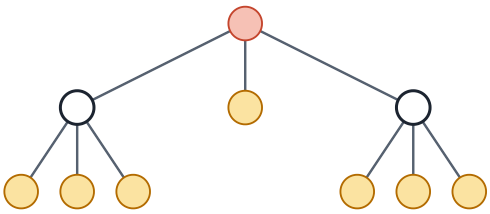
8.3 Árboles m-arios

Generalizamos la idea: en vez de permitir a lo sumo 2 hijos por vértice, permitimos a lo sumo *m*.

Definición: sea `T` un árbol con raíz y *m* un entero positivo. `T` es un **árbol m-ario** si `gs(v) ≤ m` para todo vértice *v* (cada vértice tiene a lo sumo *m* hijos). Si `m = 2`, el árbol es un **árbol binario**.

8.4 Árboles m-arios completos

Si `gs(v) = 0` ó `gs(v) = m` para todo vértice *v*, `T` es un **árbol m-ario completo**: cada vértice interno tiene exactamente *m* hijos y ninguna hoja tiene hijos. El caso especial `m = 2` es el **árbol binario completo**.



Árbol 3-ario completo: 3 vértices internos, 7 hojas, 10 vértices en total.

TEOREMA

Sea `T = (V, E)` un árbol *m*-ario completo con `|V| = n`. Si `T` tiene `ℓ` hojas e *i* vértices internos, entonces:

- `n = m·i + 1`
- `ℓ = (m − 1)·i + 1`
- `i = (ℓ − 1)/(m − 1) = (n − 1)/m`

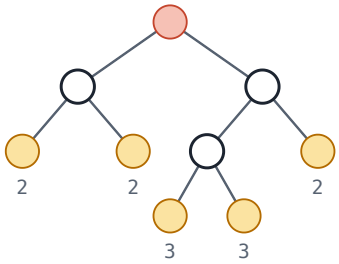
EJEMPLO — APLICACIÓN (se resuelve en clase)

Una aula tiene 25 microcomputadores que deben conectarse a un enchufe de pared con cuatro salidas. Se hacen las conexiones mediante cables de extensión con cuatro salidas cada uno. ¿Cuál es el número mínimo de cables que se necesitan para poder utilizar todos los computadores?

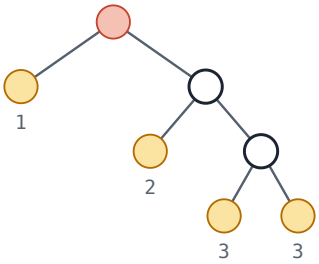
8.5 Árbol equilibrado

Si `T = (V, E)` es un árbol con raíz y *h* es el número de nivel máximo de una hoja de `T`, entonces `T` tiene **altura** *h*.

Definición: un árbol con raíz `T` de altura *h* es **equilibrado** si el número de nivel de cada hoja de `T` es `h − 1` ó `h`.



Equilibrado (altura 3): hojas en los niveles 2 y 3



No equilibrado (altura 3): hay una hoja en el nivel 1

El número bajo cada hoja es su nivel.

9. Árboles binarios

Propiedades de los árboles binarios (definidos en la sección anterior), que se usan en las secciones siguientes.

Propiedad: un árbol binario completo con *i* nodos internos tiene exactamente `i + 1` nodos terminales (hojas). Es el caso `m = 2` del teorema de la sección anterior.

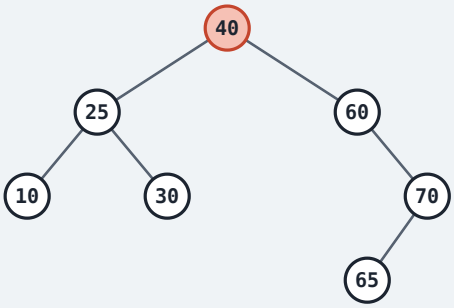
> TEOREMA
>
> En un árbol binario con *t* hojas y altura *h*: $h \geq \log_2(t)$

10. Árbol binario de búsqueda

Definición: un **árbol binario de búsqueda** (ABB) es un árbol binario en el que cada nodo guarda un dato (de un conjunto ordenado: números, palabras, ...) de manera que, para todo nodo *v*:

- todo dato del **subárbol izquierdo** de *v* es **menor** que el dato de *v*, y
- todo dato del **subárbol derecho** de *v* es **mayor** que el dato de *v*.

EJEMPLO — se insertan, en este orden: 40, 25, 60, 10, 30, 70, 65



El primer dato queda en la raíz. Cada dato nuevo se compara desde la raíz: si es menor se baja a la izquierda, si es mayor a la derecha, hasta encontrar un lugar vacío. Por ejemplo, 65 > 40 (derecha), 65 > 60 (derecha), 65 < 70 (izquierda) → queda como hijo izquierdo de 70.

Construcción (inserción)

```
Entrada: sucesión w₁, w₂, ..., wₙ de datos distintos
Salida:  T árbol binario de búsqueda

Inicio: T = árbol con un solo nodo (la raíz), que guarda w₁
Para i = 2 hasta n hacer:
  v = raíz
  busca = verdadero
  Mientras busca hacer:
    Si wᵢ < dato(v) entonces
      Si v no tiene hijo izquierdo: agregar hijo izquierdo con wᵢ;  busca = falso
      Si no: v ← hijo izquierdo de v
    Si no  (wᵢ > dato(v))
      Si v no tiene hijo derecho: agregar hijo derecho con wᵢ;  busca = falso
      Si no: v ← hijo derecho de v
Devolver T
```

Búsqueda de un dato D

```
v = raíz
Mientras v exista hacer:
  Si D = dato(v) entonces devolver v        // encontrado
  Si D < dato(v) entonces v ← hijo izquierdo de v
  Si no                  v ← hijo derecho de v
Devolver "D no está en el árbol"
```

El tiempo de búsqueda en el peor caso es proporcional a la **altura** del árbol. Usando el teorema de árboles binarios ($h \geq \log_2 t$), en un ABB con n datos el peor caso necesita al menos $\lceil \log_2(n+1) \rceil$ comparaciones, y ese valor se alcanza si el árbol tiene altura mínima.

> OJO CON EL ORDEN
> La forma del árbol depende del orden de inserción. Si los datos llegan ya ordenados (10, 25, 30, 40, ...), el ABB queda como una "cadena" y la búsqueda deja de ser rápida.

11. Recorridos de árboles binarios

Recorrer un árbol es visitar ("procesar") **cada nodo exactamente una vez** de forma sistemática. BFS y DFS son recorridos; para árboles binarios hay tres recorridos recursivos clásicos, que se distinguen por **en qué momento se procesa la raíz**:

> PREORDEN
> **raíz** · izquierdo · derecho

```
preorden(PT):
  Si PT está vacío:
    volver
  procesar PT
  preorden(hijo izq. de PT)
  preorden(hijo der. de PT)
```

> INORDEN (ENTREORDEN)
> izquierdo · **raíz** · derecho

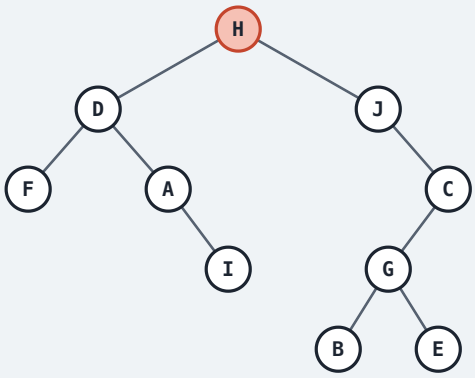
```
inorden(PT):
  Si PT está vacío:
    volver
  inorden(hijo izq. de PT)
  procesar PT
  inorden(hijo der. de PT)
```

> POSTORDEN
> izquierdo · derecho · **raíz**

```
postorden(PT):
  Si PT está vacío:
    volver
  postorden(hijo izq. de PT)
  postorden(hijo der. de PT)
  procesar PT
```

PT es la raíz del (sub)árbol. "Procesar" depende del problema: imprimir el dato, sumarlo, evaluarlo, etc.

EJEMPLO




```
Preorden: H D F A I J C G B E

Inorden: F D A I H J B G E C

Postorden: F I A D B E G C J H
```

Este árbol no es un árbol binario de búsqueda: los recorridos se pueden aplicar a cualquier árbol binario, tenga o no sus datos ordenados.

PROPIEDAD

En un **árbol binario de búsqueda**, el recorrido **inorden** procesa los datos **ordenados de menor a mayor**. En el ABB de la sección anterior: 10, 25, 30, 40, 60, 65, 70.

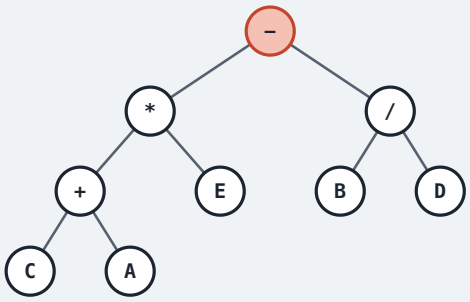
12. Árboles de expresión aritmética

Una expresión con operadores binarios `+ − * /` se puede representar con un árbol binario:

- las **hojas** son los **operandos** (variables o números);
- los **nodos internos** son los **operadores**, y cada operador actúa sobre su subárbol izquierdo y su subárbol derecho.

La distinción izquierdo/derecho importa: `4 − 6 ≠ 6 − 4` .

EJEMPLO — (C + A) * E − B / D



El operador que se aplica "último" (−) queda en la raíz.

Cada recorrido del árbol da una forma distinta de escribir la misma expresión:

- **Inorden** (poniendo paréntesis en cada operación) → **forma infija** con paréntesis: `(((C + A) * E) − (B / D))`
- **Preorden** → **forma prefija** o *notación polaca*: `− * + C A E / B D`
- **Postorden** → **forma posfija** o *notación polaca inversa*: `C A + E * B D / −`

En las formas prefija y posfija **no hacen falta paréntesis** ni reglas de prioridad entre operaciones: la expresión se evalúa sin ambigüedad. Por eso muchos compiladores traducen las expresiones a forma posfija, y algunas calculadoras trabajan directamente con ella.

Evaluar una expresión posfija (con una pila)

```
Para cada símbolo de la expresión, de izquierda a derecha:
  Si es un operando: apilarlo
  Si es un operador ∘: desapilar y, luego x; apilar (x ∘ y)
Al terminar, el resultado es el único valor de la pila
```

EJEMPLO — C A + E * B D / − con A=2, B=8, C=3, D=4, E=5

3, 2 → + → 5 · 5 → * → 25 · 8, 4 → / → 2 · 25, 2 → − → **23**

13. Árboles de decisión

Un **árbol de decisión** es un árbol con raíz que modela un proceso en el que, en cada paso, se hace una prueba o comparación con varios resultados posibles. Cada vértice interno es una prueba (con un hijo por cada resultado posible) y cada hoja es una conclusión final.

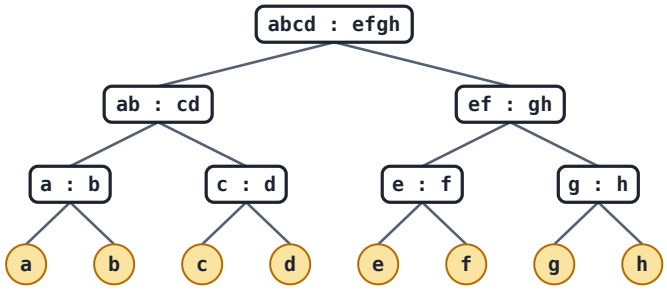
EJEMPLO — LA MONEDA FALSA

Dadas ocho monedas (idénticas en apariencia) y una balanza, si exactamente una de ellas es falsa y más pesada que las otras siete, hay que identificar la moneda falsa. Llamamos *a, b, c, d, e, f, g, h* a las monedas.

Al comparar conjuntos de monedas en la balanza hay tres resultados posibles: (1) los dos platillos quedan equilibrados, y ninguna de las monedas pesadas es falsa; (2) el platillo izquierdo queda más bajo, y la moneda falsa está en ese platillo; (3) el platillo derecho queda más bajo, y la moneda falsa está en ese platillo.

Árbol binario de decisión (altura 3)

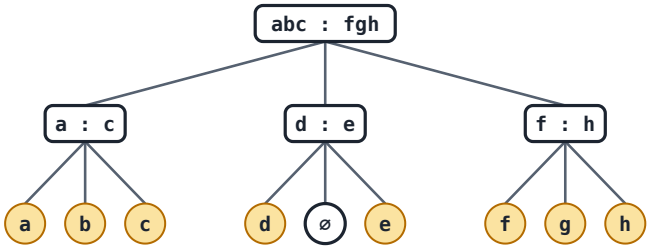
Se compara primero un grupo de 4 monedas contra otro de 4. Como hay exactamente una moneda falsa, los platillos nunca se equilibran: solo hay dos resultados, y en cada nivel se revisa la mitad de las monedas candidatas.



En cada vértice, *x : y* indica los conjuntos que se comparan. Rama izquierda: baja el platillo izquierdo. Rama derecha: baja el derecho.

Árbol ternario de decisión (altura 2)

Se compara primero un grupo de 3 monedas contra otro de 3, dejando dos monedas afuera. Ahora hay tres resultados posibles: la rama izquierda (baja el platillo izquierdo), la central (se equilibran: la falsa es una de las dos monedas que quedaron afuera) y la derecha (baja el platillo derecho). La moneda falsa se encuentra en dos pesadas.



Rama izquierda: baja el platillo izquierdo. Rama central: se equilibran. Rama derecha: baja el platillo derecho. La hoja ∅ no corresponde a ninguna moneda: es un resultado imposible.

La cantidad de pesadas que se necesitan en el peor caso es la **altura** del árbol. Como cada pesada tiene 3 resultados posibles, con *k* pesadas se distinguen a lo sumo 3^k casos. Hay 8 monedas que podrían ser la falsa: una pesada no alcanza ($3^1 = 3 < 8$), pero dos sí ($3^2 = 9 \geq 8$). Con *n* monedas hacen falta al menos $\lceil \log_3 n \rceil$ pesadas.

Bibliografía

Grimaldi, R. P. (1998). *Matemáticas discreta y combinatoria: una introducción con aplicaciones* (3.ª ed.). Addison Wesley Iberoamericana. Cap. 12, “Árboles”.

Johnsonbaugh, R. (2005). *Matemáticas discretas* (6.ª ed.; M. A. González Osuna, trad.). Pearson Educación. Cap. 9, “Árboles”.

Repaso teórico — sin demostraciones